Process & Decision Documentation

Side Quest Week 6 - Fox Rescue

Project/Assignment Decisions

I decided to combine both sound and physics feedback for the bonus requirement rather than implementing only one. The previous sketch already had physics (gravity, knockback, collisions via p5play/Planck.js), so I focused on layering sound effects and visual feedback on top of the existing mechanics. One key decision was reducing the fire collider height after playtesting revealed the player could not jump over fire hazards without losing health, which made the game too frustrating to finish. I also fixed the boar-fire death logic because boars were getting stuck in a frozen state instead of properly disappearing when they walked into fire.

Role-Based Process Evidence

Name: Jimin Kim, j243kim, 21062367 **Role(s):** Developer (individual Side Quest) **Primary responsibility for this work:** All development, testing, and iteration

Goal of Work Session

Refine the existing Fox Rescue platformer sketch by adding sound effects and physics-driven visual feedback (screen shake, particle effects) to satisfy the Week 6 requirement of adding sound or simple physics to the sketch, inspired by Papers, Please.

Tools, Resources, or Inputs Used

- Claude Code (Claude Opus 4.6) for code generation and debugging
- Prior sketch code from Week 5
- p5.js / p5play / Planck.js documentation
- Free audio assets from itch.io [7][8]
- Free music from itch.io [6]

GenAI Documentation

Date Used: 2026-03-08

Tool Disclosure: Claude Code (Claude Opus 4.6)

Purpose of Use: Used Claude Code to assist with implementing sound loading, screen shake, particle effects, and wiring event-driven audio/visual feedback into the existing codebase. Also used for debugging the boar-fire death state machine and adjusting fire collider sizing.

Summary of Interaction: Claude Code read and analyzed the existing project structure and source files, then implemented the following changes: created a ParticleSystem class for visual feedback, added screen shake to CameraController, loaded sound files in the boot pipeline, wired gameplay events to

sound/particle/shake responses, and fixed the boar-fire death interaction. It also created the README and helped iterate on fire collider sizes after I reported the game was too hard.

Human Decision Point(s):

- I chose to use Papers, Please as the inspiration source (not One Night Ultimate Werewolf) because the reactive feedback theme fit better with the existing platformer mechanics.
- I requested the fire collider be reduced further after the first reduction was still too large based on my playtesting.
- I requested the background music volume be increased from 25% to 35% after testing in-browser.
- I identified the boar-fire bug (boars freezing in place) during gameplay and directed Claude to fix it.

Integrity & Verification Note: I tested all changes by running the sketch in Chrome via a local server, verifying that sounds played on the correct events, particles appeared at the right positions, screen shake felt appropriate, and the fire collider allowed jumping over. I confirmed the boar death animation played correctly after the fix.

Scope of GenAI Use: Claude Code wrote the implementation code (ParticleSystem.js, SoundManager updates, CameraController shake, main.js event wiring, boar-fire fix). I directed all design decisions, identified bugs through playtesting, and verified all functionality in-browser. The original project architecture and game design were established in previous weeks.

Limitations or Misfires:

- The initial fire collider reduction (18x16 to 14x10) was not enough; I had to request a second reduction (to 12x6) after testing.
- Claude initially set music volume at 25% which was too quiet; I adjusted to 35%.

Summary of Process (Human + Tool)

1. Started with the existing Week 5 Fox Rescue sketch that had physics, enemies, and collectibles but no sound or visual feedback effects.
2. Used Claude Code to implement sound loading (jump, leaf collect, damage, hit enemy) and background music.
3. Added screen shake to the camera controller, triggered by player damage and boar hits.
4. Created a particle system with different burst types for leaf collection (green sparkles), boar hits (red impact), and player damage (red burst).
5. Playtested and found two issues: fire colliders were too tall to jump over, and boars froze in place when walking into fire.
6. Iterated on fire collider size twice (18x16 -> 14x10 -> 12x6) until the game felt fair.
7. Fixed boar-fire death by skipping the grounded wait so boars transition to the death animation immediately.

Decision Points & Trade-offs

1. **Fire collider sizing:** I considered keeping the original 18x16 collider for difficulty, but playtesting showed it was unfair since the player physically could not clear the fire even with a well-timed jump. I reduced it to 12x6 so the hitbox only covers the base of the flame, making jumps possible while still punishing careless movement through fire.

2. **Combining sound + physics for bonus:** Rather than implementing sound and physics as separate, unrelated features, I wired them to the same gameplay events so they feel like one connected interaction (e.g., hitting a boar produces knockback + hit sound + particles + shake simultaneously). This made the bonus requirement feel cohesive rather than tacked on.

Verification & Judgement

- Playtested the sketch multiple times in Chrome to verify all sounds trigger correctly, particles appear at the right positions, and screen shake intensity feels appropriate.
- Re-read the Week 6 assignment criteria to confirm both base (sound or physics) and bonus (both combined) requirements were met.
- Tested the fire jump specifically to ensure the reduced collider allows clearing fires with a well-timed jump.

Limitations, Dead Ends, or Open Questions

- No win jingle sound was added because no suitable sound file was available in the existing assets.
- The particle system uses simple colored rectangles rather than sprite-based particles, which is functional but visually basic.
- Background music restarts from the beginning on level restart rather than continuing seamlessly.

Appendix

Full GenAI transcript from Claude Code (Claude Opus 4.6) session on 2026-03-08.

Prompt 1 (User)

Based on the Feb26th, sidequest 06 please modify the code like the below to satisfy all of the requirements and rubric please, plus bonus mark as well.

- Refine a previous sketch
  - do not make a completely unrelated new project
  - build on one of your earlier p5.js sketches and upgrade it for Week 06
- Choose only one inspiration source
  - use inspiration from either the video game or the board game for that week
  - do not try to combine both inspirations in the assignment response
- Add at least one Week 06 mechanic
  - add sound or simple physics to the sketch
  - one of these is enough for the base requirement, but it has to be clearly present and functional
- Make the mechanic triggered by gameplay
  - the effect should happen because of: motion, collisions, player actions
  - it should not just sit there passively in the background with no gameplay connection
- If you do sound for the base requirement
  - load at least one sound file
  - trigger it from an actual event in the sketch
  - make sure it plays reliably in browser during actual play
- If you do simple physics for the base requirement

    - add a clear physical behavior such as gravity, jumping/falling, bouncing, velocity, acceleration, knockback, friction/slowdown
    - the motion should feel rule-based, not random decoration
- Add visible feedback
    - examples: flash on hit, bounce effect, particles, squash/stretch, shake, animation on impact
- Make the interaction clear and playable
- Keep visuals coherent
- Go a little beyond bare minimum

For the bonus point, do all of the above and also:

- Combine both sound and physics
- Integrate them together
- Make the bonus fully functional

[Full detailed requirements were provided in the prompt]

Response 1 (Claude Code)

Let me first look at the current project structure and existing code.

[Claude read and analyzed all project files: main.js, index.html, Game.js, SoundManager.js, PlayerController.js, Level.js, PlayerEntity.js, AssetLoader.js, tuning.json, HUDRenderer.js, BoarSystem.js, CameraController.js, InputManager.js, Leaf.js, WinScreen.js, LoseScreen.js]

Now I have a thorough understanding of the codebase. The project already has physics (gravity, knockback, collisions) and sound infrastructure, but sounds are never actually loaded. I need to:

1. Load sound files in boot
2. Add screen shake on damage
3. Add particle effects for visual feedback
4. Wire all sounds to gameplay events
5. Add background music

Files created:

- `src/ParticleSystem.js` — Particle effects system with `sparkle()`, `impact()`, and `damageBurst()` methods

Files modified:

- `src/SoundManager.js` — Added `loadMusic()`, `startMusic()`, `stopMusic()` and `isPlaying()` check
- `src/CameraController.js` — Added `shake(intensity, duration)` with decaying random offset
- `main.js` — Loaded all 4 SFX files + background music, created ParticleSystem, wired event listeners for reactive feedback (damage shake, death shake, leaf sparkles, boar impact particles, jump sound)
- `src/Game.js` — Removed references to nonexistent "die" and "win" sound keys

---

Prompt 2 (User)

did you put the background sound as well?

Response 2 (Claude Code)

Yes, background music is loaded and wired. In main.js:145:

```
soundManager.loadMusic("assets/sfx/music.wav", 0.25);
```

And it starts automatically after the first user gesture (click/keypress) in main.js:302:

```
if (audioUnlocked) soundManager?.startMusic();
```

The music loops continuously at 25% volume using the `SoundManager.startMusic()` method, which calls `.loop()` on the loaded audio. It stops and restarts on level restart via the `level:restarted` event.

Prompt 3 (User)

[User provided a screenshot of the game running]

Could you increase the background sound to 35%? Also, when an enemy goes into the fire, it currently just stays there in a fixed position, so could you make it disappear instead? Lastly, the fire barrier feels a bit too high because even when the character jumps, it still loses a heart, which makes the game too hard to finish. Could you adjust that as well?

Response 3 (Claude Code)

Now I understand all three issues. Let me fix them:

1. **Music volume**: 0.25 → 0.35
2. **Boar stuck in fire**: The boar enters `dying` state but waits for `grounded` before transitioning to `dead`. When fire kills it, it gets stuck. Fix: skip the grounded wait for fire kills.
3. **Fire collider too tall**: Reduce collider height so the player can jump over.

Changes made:

`main.js` line 145:

```
soundManager.loadMusic("assets/sfx/music.wav", 0.35);
```

`src/Level.js` — boar-fire overlap callback updated:

```
this.boar.overlaps(this.fire, (boarSprite) => {
  if (boarSprite.dead || boarSprite.dying) return;
  boarSprite.hp = 0;
```

```
    boarSprite.dying = true;
    // Skip the grounded wait — fire kills instantly so the boar doesn't get
  stuck
    boarSprite.dead = true;
    boarSprite.deathStarted = false;
    boarSprite.knockTimer = 0;
    boarSprite.vel.x = 0;
    boarSprite.vel.y = 0;
  });
```

Fire collider reduced in `src/world/TileBuilder.js`, `src/entities/Fire.js`, and `data/tuning.json`:

```
w: 18, h: 16  →  w: 14, h: 10
```

Prompt 4 (User)

still the fire collider is little bit high, please reduce it

Response 4 (Claude Code)

Reduced the fire collider from 14x10 to 12x6 across all three files (`src/world/TileBuilder.js`, `src/entities/Fire.js`, `data/tuning.json`). The hitbox is now much shorter — should give plenty of clearance when jumping over.